

AppNotas

Eva Gallardo Romero

26 de noviembre de 2025 a 11 de noviembre de 2026

Índice

Enlace al repositorio de GitHub **3**

1. Descripción general **3**

2. Diagrama de clases **3**

3. Diseño **4**
3.1. Logo: la tortuga . . . 4
3.2. Pantalla de bienvenida . . . 4
3.3. Registro por pasos . . . 5
3.4. Pantalla general . . . 7
3.5. Menú lateral . . . 8
3.6. Escribir y editar una nota . . . 8
3.7. Opciones . . . 9
3.8. Nota aleatoria . . . 10
3.9. Colores de las notas . . . 11

4. Requisitos del sistema **12**
4.1. Requisitos funcionales . . . 12
4.2. Requisitos no funcionales . . . 13

5. Documentación técnica **13**
5.1. ColorNota.java . . . 13
5.1.1. Ubicación . . . 13
5.1.2. Funciones . . . 14
5.2. Nota.java . . . 14
5.2.1. Ubicación . . . 14
5.2.2. Funciones . . . 14
5.3. Usuario.java . . . 15
5.3.1. Ubicación . . . 15
5.3.2. Funciones . . . 15
5.4. NotaManager.java . . . 16
5.4.1. Ubicación . . . 16
5.4.2. Funciones . . . 16
5.5. LandingViewModel.java . . . 18
5.5.1. Ubicación . . . 18
5.5.2. Funciones . . . 18
5.6. LandingActivity.java . . . 20
5.6.1. Ubicación . . . 20
5.6.2. Funciones . . . 21
5.7. LandingNombreFragment.java . . . 21
5.7.1. Ubicación . . . 21
5.7.2. Funciones . . . 22
5.8. LandingContrasenaFragment.java . . . 22
5.8.1. Ubicación . . . 22

5.8.2. Funciones 22
5.9. LandingFechaFragment.java 23
5.9.1. Ubicación 23
5.9.2. Funciones 23
5.10. LandingFinalFragment.java 24
5.10.1. Ubicación 24
5.10.2. Funciones 24
5.11. GeneralActivity.java 24
5.11.1. Ubicación 24
5.11.2. Funciones 24
5.12. CrearNotasActivity.java 27
5.12.1. Ubicación 27
5.12.2. Funciones 27
5.13. OpcionesActivity.java 29
5.13.1. Ubicación 29
5.13.2. Funciones 29
5.14. NotaAleatoriaActivity.java 31
5.14.1. Ubicación 31
5.14.2. Funciones 31
5.15. NotaAdapter.java 32
5.15.1. Ubicación 32
5.15.2. Funciones 32

6. Mejoras futuras **33**

Enlace al repositorio de GitHub

Puedes consultar el proyecto completo y descargable en el siguiente enlace: App-Notas.

La aplicación instalable (APK) está en la sección Releases y el diseño, en Figma.

1. Descripción general

AppNotas es una aplicación móvil para Android, hecha en **Java**, para escribir y organizar **notas de colores**. La primera vez que se abre, el usuario se **registra por pasos** (nombre, contraseña y año de nacimiento) y, a partir de ahí, entra directamente a su lista de notas.

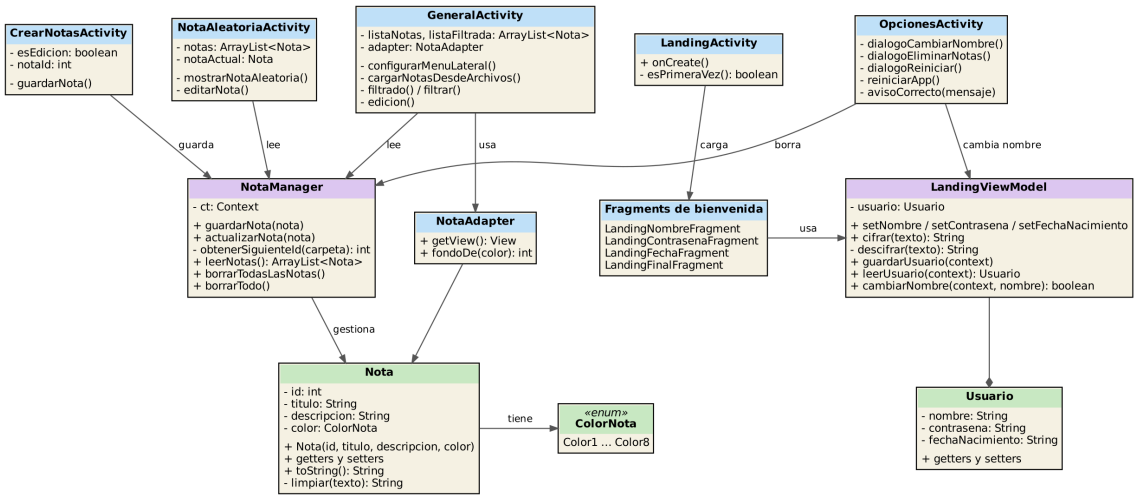
Cada nota tiene **título**, **texto** y uno de **siete colores pastel**, y se guarda en un **archivo local** dentro del almacenamiento interno de la app, así que funciona **sin conexión**. La lista se puede **filtrar por uno o varios colores** y cualquier nota se puede **editar** tocándola.

Desde el **menú lateral** se accede a escribir una nota, a las **opciones** (cambiar el nombre, eliminar todas las notas o reiniciar la app) y a una **nota aleatoria**. La mascota de la app es una **tortuga**, que aparece en la bienvenida, en el menú y en los avisos.

La primera versión (bienvenida, crear, editar y filtrar notas) la hice entre el 26 y el 29 de noviembre de 2025. En noviembre de 2026 terminé lo que faltaba del diseño: el menú lateral, las opciones, la nota aleatoria, los botones de Material 3, los colores pastel y el icono.

2. Diagrama de clases

En este punto elaboré un **diagrama de clases** para definir las **relaciones** entre las principales partes de la aplicación. El código sigue una separación parecida a **MVVM**: el **modelo** (`Model`), las clases que **gestionan los datos** (`ViewModel`) y las **pantallas** (`View`). No incluye los getters y setters uno a uno.



3. Diseño

En esta sección muestro cómo es cada pantalla de la app. El diseño lo hice primero en Figma, con una cabecera **verde**, un fondo **crema** y botones con forma de **píldora**.

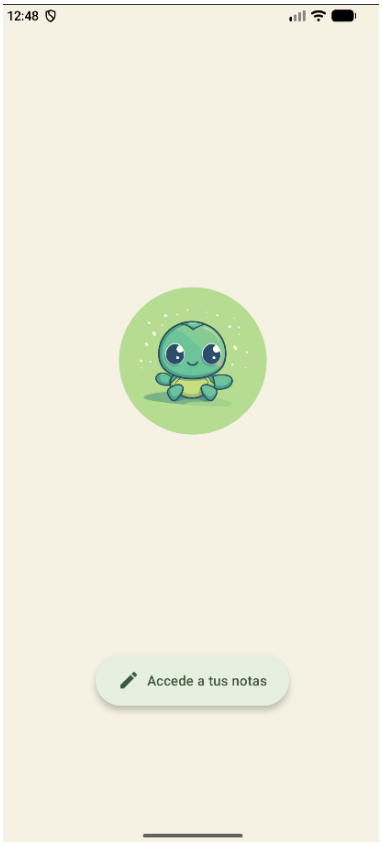
3.1. Logo: la tortuga

Quise que la app tuviera una **mascota**: una tortuga sencilla y amable sobre un fondo verde claro. Es también el **icono** de la app.



3.2. Pantalla de bienvenida

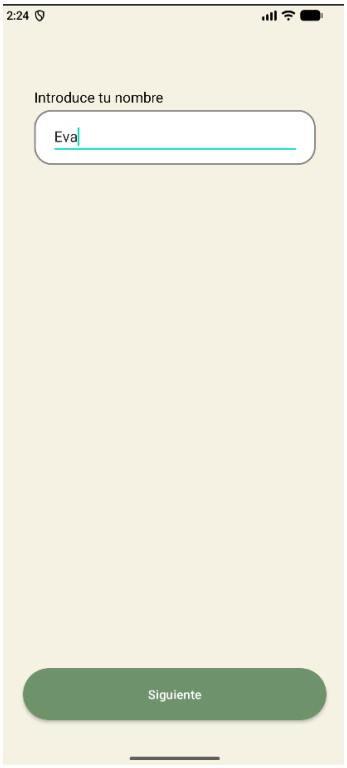
Es lo primero que ve el usuario al abrir la aplicación: la tortuga y un botón elevado, «*Accede a tus notas*».

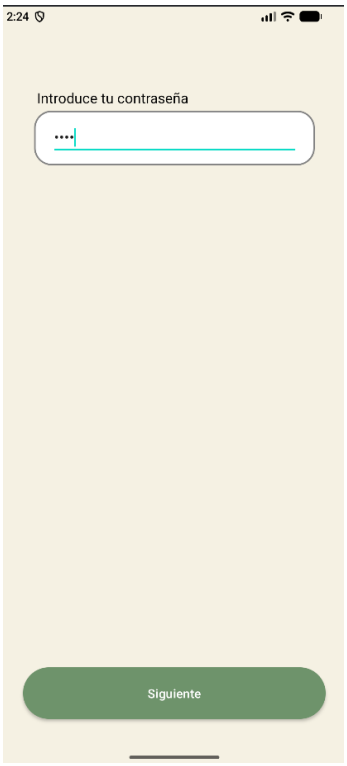


Si es la **primera vez**, el botón abre el registro. Si el usuario ya se registró, lleva directamente a la **lista de notas**.

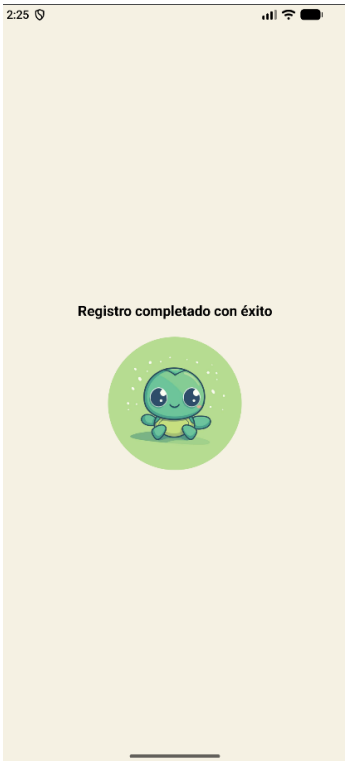
3.3. Registro por pasos

El registro está hecho con **fragments** que se van sustituyendo dentro de un mismo contenedor: primero el **nombre**, después la **contraseña** y, por último, el **año de nacimiento**. Ningún campo puede quedar vacío; si lo está, la app avisa con un `Toast`.





Al terminar se muestra «*Registro completado con éxito*» con la tortuga y, a los 3 segundos, la app pasa a la pantalla general.

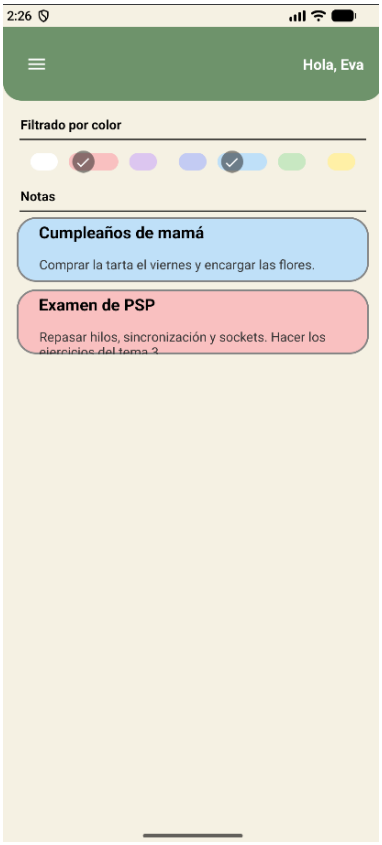


3.4. Pantalla general

Es la pantalla principal. En la cabecera está el **icono del menú** y el **saludo** con el nombre del usuario. Debajo, el **filtro por color** y la **lista de notas**, de la más nueva a la más antigua, cada una con el fondo de su color.



El filtro funciona con **chips**: se pueden marcar uno o varios colores y la lista muestra solo esas notas. Si no hay ninguno marcado, o están todos, se ven todas. Si no queda ninguna nota, aparece un aviso.

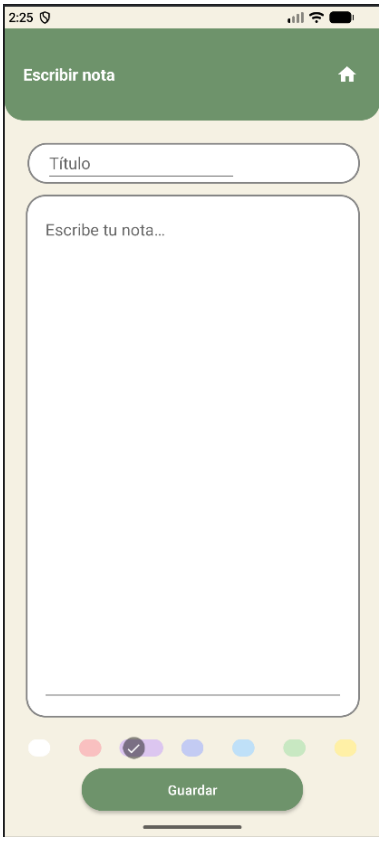


3.5. Menú lateral

El icono de la cabecera abre un `NavigationView` con la tortuga, el saludo y tres opciones: **Escribir nota**, **Opciones** y **Nota aleatoria**. Con el botón atrás se cierra el menú.

3.6. Escribir y editar una nota

El formulario tiene el **título**, el **texto** y una fila de **chips** para elegir el color. Al pulsar **Guardar**, la nota se crea y los campos se vacían para escribir otra. Si se ha abierto tocando una nota de la lista, los campos aparecen **rellenos**, la cabecera dice «*Editar nota*» y al guardar se **actualiza** esa nota. El icono de la casa vuelve a la lista.

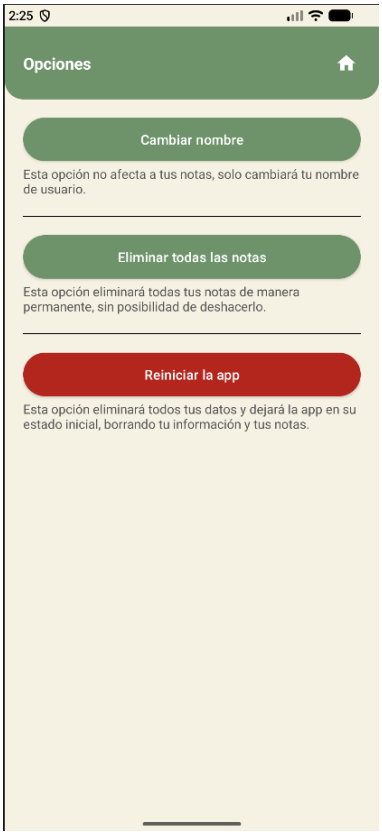


3.7. Opciones

Reúne tres acciones, cada una con una explicación debajo:

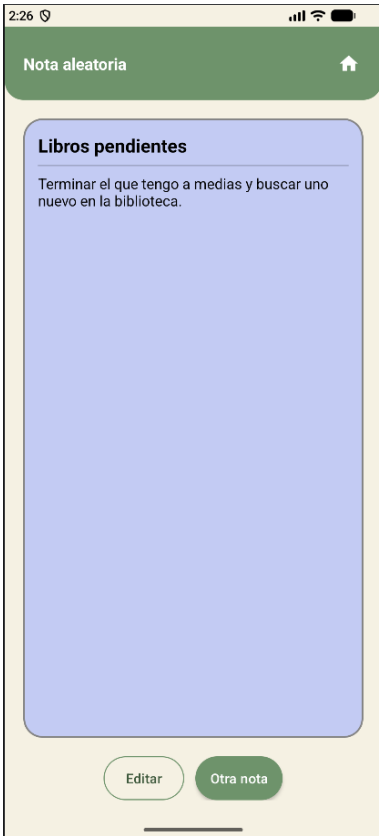
- **Cambiar nombre**: pide el nombre nuevo en un diálogo. No afecta a las notas.
- **Eliminar todas las notas**: pide confirmación porque no se puede deshacer.
- **Reiniciar la app**: en rojo, porque borra todos los datos y vuelve a la bienvenida.

Cuando la acción termina, un aviso con la tortuga confirma que todo ha ido bien.



3.8. Nota aleatoria

Muestra una de las notas elegida al azar, con su color. **Otra nota** elige otra distinta y **Editar** la abre en el formulario.



3.9. Colores de las notas

Elegí **tonos pastel** para que las notas sean suaves a la vista y el texto negro se lea bien sobre todos ellos.

Valor	Color	Código
`Color1`	Blanco	`#FFFFFF`
`Color2`	Rojo	`#F9C0C0`
`Color3`	Morado	`#DCC6F0`
`Color4`	Índigo	`#C3CBF3`
`Color5`	Azul	`#BFE0F8`
`Color6`	Verde	`#C8E8C2`
`Color7`	Amarillo	`#FFF0A6`

4. Requisitos del sistema

4.1. Requisitos funcionales

ID	Requisito funcional
RF01	La aplicación debe mostrar una **pantalla de bienvenida** con un botón de acceso.
RF02	La primera vez, el usuario debe **registrarse por pasos** con nombre, contraseña y año de nacimiento, sin dejar campos vacíos.
RF03	El sistema debe **guardar los datos del usuario** en el archivo `USUARIO/datos.cas`.
RF04	La aplicación debe **recordar que el usuario ya se registró** y entrar directamente a la lista.
RF05	La pantalla general debe **saludar al usuario** por su nombre y mostrar la **lista de notas**.
RF06	El usuario debe poder **crear una nota** con título, texto y uno de siete colores.
RF07	El usuario debe poder **editar una nota** tocándola en la lista.
RF08	Cada nota se debe guardar en su **propio archivo** `nota<ID>.not`, con un **ID incremental**.
RF09	El usuario debe poder **filtrar las notas por uno o varios colores** mediante chips.
RF10	El sistema debe mostrar un **aviso** cuando no haya notas que mostrar.
RF11	La aplicación debe tener un **menú lateral** con Escribir nota, Opciones y Nota aleatoria.
RF12	El usuario debe poder **cambiar su nombre** sin perder sus notas.
RF13	El usuario debe poder **eliminar todas sus notas**, previa confirmación.
RF14	El usuario debe poder **reiniciar la app** a su estado inicial, previa confirmación.
RF15	La aplicación debe mostrar una **nota aleatoria**, con opción de ver otra o editarla.

4.2. Requisitos no funcionales

ID	Requisito no funcional
RNF01	La interfaz debe tener un **diseño claro e intuitivo**, fiel al prototipo de Figma.
RNF02	La aplicación debe funcionar en **modo local**, sin conexión a internet.
RNF03	Los datos se deben guardar en el **almacenamiento interno privado** de la app.
RNF04	Las acciones que no se pueden deshacer deben **pedir confirmación**.
RNF05	El texto de las notas debe tener una **legibilidad adecuada** sobre todos los colores.
RNF06	La aplicación debe dar **feedback inmediato** con mensajes `Toast` y diálogos.
RNF07	El sistema debe **manejar los errores de lectura y escritura** de archivos sin cerrarse.
RNF08	El código debe estar **separado en capas** (modelo, gestión de datos y vistas).
RNF09	Las pantallas deben **adaptarse a distintos tamaños** de móvil.
RNF10	La aplicación debe funcionar en **Android 7.0 (API 24)** o superior.

5. Documentación técnica

Este anexo agrupa la documentación por archivos, separándola en la **ubicación** donde se encuentra, una **descripción general** y las **funciones** explicadas de manera individual con su **código** adjunto y comentado.

5.1. ColorNota.java

5.1.1. Ubicación

`com.example.appnotas.Model`

Descripción general:
El enum `ColorNota` representa los colores que puede tener una nota. Cada valor (`Color1` a `Color8`) se corresponde con un color de `colors.xml` y con un fondo redondeado en `drawable`. En la app se usan los siete primeros, en tonos pastel.

5.1.2. Funciones

Valores **Descripción:** Lista de colores disponibles. Se guardan por su nombre en el fichero de cada nota.

Código comentado:

```
public enum ColorNota {
    Color1,
    Color2,
    Color3,
    Color4,
    Color5,
    Color6,
    Color7,
    Color8
}
```

5.2. Nota.java

5.2.1. Ubicación

`com.example.appnotas.Model`

Descripción general:
La clase `Nota` es el modelo de cada nota: un identificador, un título, una descripción y un color. Además sabe convertirse en la línea de texto que se guarda en su fichero.

5.2.2. Funciones

Constructor **Descripción:** Crea una nota con todos sus atributos.

Código comentado:

```
public Nota(int id, String titulo, String descripcion, ColorNota color) {
    this.id = id;
    this.titulo = titulo;
    this.descripcion = descripcion;
    this.color = color;
}
```

Getters **Descripción:** Métodos para consultar los atributos de la nota.

Código comentado:

```
public int getId() {
    return id;
}
public String getTitulo() {
    return titulo;
}
public String getDescripcion() {
    return descripcion;
}
public ColorNota getColor() {
    return color;
}
```

toString **Descripción:** Devuelve la nota en el formato del fichero: `titulo | descripcion | color | id`, todo en una línea.

Código comentado:

```
// Formato del fichero: titulo | descripcion | color | id (todo en una línea).
// Los saltos de línea se guardan como "\\n" para que una descripción de varias
// líneas no rompa la lectura, y la barra "|" se cambia por "/" porque es el separador.
@Override
public String toString() {
    return limpiar(titulo) + " | " +
            limpiar(descripcion) + " | " +
            color + " | " +
            id;
}
```

limpiar **Descripción:** Prepara un texto para guardarlo: cambia la barra `|` (el separador) por `/` y los saltos de línea por `\n`, para que una descripción de varias líneas no rompa la lectura.

Código comentado:

```
private static String limpiar(String texto) {
    return texto.replace("|", "/").replace("\r", "").replace("\n", "\\n");
}
```

5.3. Usuario.java

5.3.1. Ubicación

`com.example.appnotas.Model`

Descripción general:
La clase `Usuario` guarda los datos que se piden en la bienvenida: nombre, contraseña y año de nacimiento.

5.3.2. Funciones

Constructor **Descripción:** Crea un usuario con sus tres datos.

Código comentado:

```
public Usuario(String nombre, String contrasena, String fechaNacimiento) {
    this.nombre = nombre;
    this.contrasena = contrasena;
    this.fechaNacimiento = fechaNacimiento;
}
```

Getters y setters **Descripción:** Métodos para consultar y cambiar cada dato.

Código comentado:

```
public String getNombre() {
    return nombre;
}
public void setNombre(String nombre) {
    this.nombre = nombre;
}
```

5.4. NotaManager.java

5.4.1. Ubicación

`com.example.appnotas.ViewModel`

Descripción general:
La clase `NotaManager` se encarga de guardar las notas en el almacenamiento interno de la app. Todas viven en la carpeta `USUARIO`, cada una en su fichero `nota<ID>.not`. También lee todas las notas y las borra desde Opciones.

5.4.2. Funciones

Constructor **Descripción:** Guarda el `Context` para poder acceder a la carpeta interna de la app.

Código comentado:

```
public NotaManager(Context ct){
    this.ct = ct;
}
```

guardarNota **Descripción:** Crea el fichero de una nota nueva, con el siguiente identificador libre.

Código comentado:

```
public void guardarNota(Nota nota){
    try{
        File carpeta = new File(ct.getFilesDir(), "USUARIO");

        int id = obtenerSiguienteId(carpeta);
        nota = new Nota(id, nota.getTitulo(), nota.getDescripcion(), nota.getColor());

        String nombreArchivo = ("nota" + id + ".not");
        File archivoNota = new File(carpeta, nombreArchivo);

        FileWriter writer = new FileWriter(archivoNota);
        writer.write(nota.toString());
        writer.close();
    }catch (IOException e ){
        e.printStackTrace();
    }
}
```

actualizarNota **Descripción:** Sobrescribe el fichero de una nota que ya existe con sus datos editados.

Código comentado:

```
public void actualizarNota(Nota nota){
    try{
        File carpeta = new File(ct.getFilesDir(), "USUARIO");

        String nombreArchivo = "nota" + nota.getId() + ".not";
        File archivoNota = new File(carpeta, nombreArchivo);

        FileWriter writer = new FileWriter(archivoNota);
        writer.write(nota.toString());
        writer.close();
    }catch (IOException e ){
```

```
12          e.printStackTrace();
13      }
14  }
```

obtenerSiguienteId **Descripción:** Calcula el identificador de una nota nueva: el mayor que ya exista más uno.

Código comentado:

```
1  // El id de una nota nueva es el mayor id que ya exista más uno (nota1.not, nota2.not...)
2  private int obtenerSiguienteId(File carpeta){
3      if (!carpeta.exists()) carpeta.mkdir();
4      File[] archivos = carpeta.listFiles((dir, nombre) -> nombre.startsWith("nota") && nombre.
       endsWith(".not"));
5      int mayor = 0;
6      if (archivos != null) {
7          for (File archivo : archivos) {
8              String numero = archivo.getName().replace("nota", "").replace(".not", "");
9              try {
10                 mayor = Math.max(mayor, Integer.parseInt(numero));
11             } catch (NumberFormatException e) {
12                 // nombre inesperado: se ignora
13             }
14         }
15     }
16     return mayor + 1;
17 }
```

leerNotas **Descripción:** Lee todos los ficheros `.not` y devuelve la lista de notas, de la más nueva a la más antigua.

Código comentado:

```
1  // Lee todas las notas guardadas (ficheros .not), de la más nueva a la más antigua
2  public ArrayList<Nota> leerNotas() {
3      ArrayList<Nota> notas = new ArrayList<>();
4      File carpeta = new File(ct.getFilesDir(), "USUARIO");
5      // Solo los ficheros de notas (.not); en la carpeta también está datos.cas con el usuario
6      File[] archivos = carpeta.listFiles((dir, nombre) -> nombre.endsWith(".not"));
7      if (archivos == null) return notas;
8
9      for (File archivo : archivos) {
10         try (BufferedReader br = new BufferedReader(new FileReader(archivo))) {
11             String linea = br.readLine();
12             if (linea == null) continue;
13             String[] partes = linea.split("\\|");
14
15             String titulo = partes[0].trim();
16             String descripcion = partes[1].trim().replace("\\n", "\n");
17             ColorNota color = ColorNota.valueOf(partes[2].trim());
18             int id = Integer.parseInt(partes[3].trim());
19
20             notas.add(new Nota(id, titulo, descripcion, color));
21         } catch (Exception e) {
22             // Fichero con un formato inesperado: se salta
23             e.printStackTrace();
24         }
25     }
26
27     // Ordena por id de mayor a menor para que la última nota salga arriba
28     Collections.sort(notas, (a, b) -> Integer.compare(b.getId(), a.getId()));
29     return notas;
30 }
```

borrarTodasLasNotas **Descripción:** Borra todas las notas, pero deja los datos del usuario.

Código comentado:

```
// Borra todas las notas, pero deja los datos del usuario (datos.cas)
public void borrarTodasLasNotas() {
    File carpeta = new File(ct.getFilesDir(), "USUARIO");
    File[] archivos = carpeta.listFiles((dir, nombre) -> nombre.endsWith(".not"));
    if (archivos == null) return;
    for (File archivo : archivos) {
        archivo.delete();
    }
}
```

borrarTodo **Descripción:** Borra las notas y los datos del usuario para dejar la app en su estado inicial.

Código comentado:

```
// Borra las notas y los datos del usuario: la app vuelve a su estado inicial
public void borrarTodo() {
    File carpeta = new File(ct.getFilesDir(), "USUARIO");
    File[] archivos = carpeta.listFiles();
    if (archivos != null) {
        for (File archivo : archivos) {
            archivo.delete();
        }
    }
    carpeta.delete();
}
```

5.5. LandingViewModel.java

5.5.1. Ubicación

`com.example.appnotas.ViewModel`

Descripción general:
La clase `LandingViewModel` recoge los datos de la bienvenida paso a paso y los guarda en el fichero `datos.cas`. Los datos se guardan con un cifrado sencillo que desplaza cada carácter una posición.

5.5.2. Funciones

Setters **Descripción:** Guardan en el `Usuario` cada dato a medida que se rellena su pantalla.

Código comentado:

```
public void setNombre(String nombre) {
    usuario.setNombre(nombre);
}
public void setContrasena(String contrasena) {
    usuario.setContrasena(contrasena);
}
public void setFechaNacimiento(String fecha) {
    usuario.setFechaNacimiento(fecha);
}
```

cifrar **Descripción:** Desplaza cada carácter del texto una posición (por ejemplo, la `a` pasa a ser `b`).

Código comentado:

```java
public String cifrar(String texto){
    char[] letras = texto.toCharArray();
    String resultado = "";
    for (int i = 0; i < texto.length(); i++){
        char letra = letras[i];
        char cifrada = (char)(letra + 1);
        resultado += cifrada;
    }
    return resultado;
}
```

guardarUsuario **Descripción:** Cifra los tres datos y los escribe en `USUARIO/datos.cas`.

Código comentado:

```java
public void guardarUsuario(Context context) {
    String nombreCifrado = cifrar(usuario.getNombre());
    String contrasenaCifrada = cifrar(usuario.getContrasena());
    String fechaCifrada = cifrar(usuario.getFechaNacimiento());

    String datos = "Nombre: " + nombreCifrado + "\n" +
                   "Contraseña: " + contrasenaCifrada + "\n" +
                   "Fecha: " + fechaCifrada + "\n";
    try {
        File carpeta = new File(context.getFilesDir(), "USUARIO");
        if(!carpeta.exists()){ carpeta.mkdir(); }

        File archivo = new File(carpeta, "datos.cas");

        FileWriter writer = new FileWriter(archivo);
        writer.write(datos.toString());
        writer.close();

        Log.d("LandingViewModel", "Usuario guardado");
    } catch (IOException e) {
        e.printStackTrace();
    }
}
```

leerUsuario **Descripción:** Lee el nombre del fichero y lo descifra para el saludo.

Código comentado:

```java
public Usuario leerUsuario(Context context) {
    File carpeta = new File(context.getFilesDir(), "USUARIO");
    File archivo = new File(carpeta, "datos.cas");

    Usuario usuario = new Usuario();

    try (BufferedReader br = new BufferedReader(new FileReader(archivo))) {
        String linea;
        while ((linea = br.readLine()) != null){
            if (linea.startsWith("Nombre:")) {
                String nombreCifrado = linea.replace("Nombre:", "").trim();
                usuario.setNombre(descifrar(nombreCifrado));
            }
        }

    } catch (Exception e) {
        e.printStackTrace();
    }

    return usuario;
```

```
21 }
```

descifrar **Descripción:** Deshace el desplazamiento de `cifrar`.

Código comentado:

```
1 private String descifrar(String texto) {
2     char[] letras = texto.toCharArray();
3     String resultado = "";
4     for (int i = 0; i < letras.length; i++) {
5         char original = (char)(letras[i] - 1);
6         resultado += original;
7     }
8     return resultado;
9 }
```

cambiarNombre **Descripción:** Cambia solo la línea del nombre en `datos.cas`; la contraseña y la fecha se quedan igual.

Código comentado:

```
1  // Cambia solo el nombre: la contraseña y la fecha se quedan como estaban en datos.cas
2  public boolean cambiarNombre(Context context, String nuevoNombre) {
3      File archivo = new File(new File(context.getFilesDir(), "USUARIO"), "datos.cas");
4      StringBuilder datos = new StringBuilder();
5
6      try (BufferedReader br = new BufferedReader(new FileReader(archivo))) {
7          String linea;
8          while ((linea = br.readLine()) != null) {
9              if (linea.startsWith("Nombre:")) {
10                 datos.append("Nombre: ").append(cifrar(nuevoNombre)).append("\n");
11             } else {
12                 datos.append(linea).append("\n");
13             }
14         }
15     } catch (IOException e) {
16         e.printStackTrace();
17         return false;
18     }
19
20     try (FileWriter writer = new FileWriter(archivo)) {
21         writer.write(datos.toString());
22         return true;
23     } catch (IOException e) {
24         e.printStackTrace();
25         return false;
26     }
27 }
```

5.6. LandingActivity.java

5.6.1. Ubicación

`com.example.appnotas.View.Landing`

Descripción general:

La clase `LandingActivity` es la primera pantalla de la app. Muestra la tortuga y el botón «Accede a tus notas». La primera vez abre el registro por pasos; las siguientes entra directamente a la lista de notas.

5.6.2. Funciones

onCreate **Descripción:** Configura la pantalla y decide qué hace el botón según sea o no la primera vez.

Código comentado:

```java
@Override
protected void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
    EdgeToEdge.enable(this);
    setContentView(R.layout.activity_landing);
    ViewCompat.setOnApplyWindowInsetsListener(findViewById(R.id.main), (v, insets) -> {
        Insets systemBars = insets.getInsets(WindowInsetsCompat.Type.systemBars());
        v.setPadding(systemBars.left, systemBars.top, systemBars.right, systemBars.bottom);
        return insets;
    });

    btnAcceder = findViewById(R.id.btnAcceder);

    fragmentContainerLanding = findViewById(R.id.fragmentContainerLanding);


    btnAcceder.setOnClickListener(v -> {
        if (esPrimeraVez()) {
            btnAcceder.setVisibility(View.GONE);
            fragmentContainerLanding.setVisibility(View.VISIBLE);

            LandingNombreFragment nombreFragment = new LandingNombreFragment();
            FragmentTransaction transaction = getSupportFragmentManager().beginTransaction();
            transaction.replace(R.id.fragmentContainerLanding, nombreFragment);
            transaction.commit();
        } else {
            Intent intent = new Intent(LandingActivity.this, GeneralActivity.class);
            startActivity(intent);
            // Se cierra la bienvenida para que el botón atrás no vuelva a ella
            finish();
        }
    });
}
```

esPrimeraVez **Descripción:** Consulta en `SharedPreferences` si el usuario ya se ha registrado.

Código comentado:

```java
private boolean esPrimeraVez(){
    SharedPreferences prefs = getSharedPreferences(PREFS_APP, MODE_PRIVATE);
    boolean esPrimeraVez = prefs.getBoolean(KEY_PRIMERA_VEZ, true);

    return esPrimeraVez;

}
```

5.7. LandingNombreFragment.java

5.7.1. Ubicación

`com.example.appnotas.View.Landing`

Descripción general:
Primer paso del registro: pide el nombre del usuario.

5.7.2. Funciones

onCreateView **Descripción:** Comprueba que el nombre no esté vacío, lo guarda en el ViewModel y pasa al paso de la contraseña.

Código comentado:

```
@Override
public View onCreateView(LayoutInflater inflater, ViewGroup container,
                         Bundle savedInstanceState) {
    View rootView = inflater.inflate(R.layout.fragment_landing_nombre, container, false);

    Button btnNombre = (Button) rootView.findViewById(R.id.btnNombre);
    EditText editNombre = (EditText) rootView.findViewById(R.id.editNombre);

    btnNombre.setOnClickListener(v ->{
        String nombre = editNombre.getText().toString();
        if (nombre.isEmpty()) {
            Toast.makeText(getContext(), "El nombre es obligatorio", Toast.LENGTH_SHORT).show();
        } else {
            viewModel.setNombre(nombre);

            LandingContrasenaFragment contrasenaFragment = new LandingContrasenaFragment(
     viewModel);
            FragmentTransaction transaction = getActivity().getSupportFragmentManager().
     beginTransaction();
            transaction.replace(R.id.fragmentContainerLanding, contrasenaFragment);
            transaction.commit();
        }
    });

    return rootView;
}
```

5.8. LandingContrasenaFragment.java

5.8.1. Ubicación

`com.example.appnotas.View.Landing`

Descripción general:
Segundo paso del registro: pide la contraseña.

5.8.2. Funciones

onCreateView **Descripción:** Comprueba que la contraseña no esté vacía y pasa al paso del año de nacimiento.

Código comentado:

```
@Override
public View onCreateView(LayoutInflater inflater, ViewGroup container,
                         Bundle savedInstanceState) {
    View rootView = inflater.inflate(R.layout.fragment_landing_contrasena, container, false);

    Button btnContrasena = (Button) rootView.findViewById(R.id.btnContrasena);
    EditText editContrasena = (EditText) rootView.findViewById(R.id.editContrasena);

    btnContrasena.setOnClickListener(v ->{
        String contrasena = editContrasena.getText().toString();
        if (contrasena.isEmpty()) {
            Toast.makeText(getContext(), "El contraseña es obligatorio", Toast.LENGTH_SHORT).show
     ();
        } else {
```

```
14              viewModel.setContrasena(contrasena);
15
16              LandingFechaFragment fechaFragment = new LandingFechaFragment(viewModel);
17              FragmentTransaction transaction = getActivity().getSupportFragmentManager().
        beginTransaction();
18              transaction.replace(R.id.fragmentContainerLanding, fechaFragment);
19              transaction.commit();
20          }
21      });
22
23      return rootView;
24  }
```

5.9. LandingFechaFragment.java

5.9.1. Ubicación

`com.example.appnotas.View.Landing`

Descripción general:
Tercer paso del registro: pide el año de nacimiento y guarda el usuario.

5.9.2. Funciones

onCreateView **Descripción:** Guarda el usuario en su fichero, marca que ya no es la primera vez y muestra el registro completado.

Código comentado:

```
1   @Override
2   public View onCreateView(LayoutInflater inflater, ViewGroup container,
3                            Bundle savedInstanceState) {
4       View rootView = inflater.inflate(R.layout.fragment_landing_fecha, container, false);
5
6       Button btnFecha = (Button) rootView.findViewById(R.id.btnFecha);
7       EditText editFecha = (EditText) rootView.findViewById(R.id.editFecha);
8
9       btnFecha.setOnClickListener(v ->{
10          String fecha = editFecha.getText().toString();
11
12          if (fecha.isEmpty()) {
13              Toast.makeText(getContext(), "El año de nacimiento es obligatorio", Toast.
        LENGTH_SHORT).show();
14          } else {
15              viewModel.setFechaNacimiento(fecha);
16              viewModel.guardarUsuario(getContext());
17              // A partir de ahora la app entra directamente a las notas
18              getContext().getSharedPreferences(LandingActivity.PREFS_APP, Context.MODE_PRIVATE)
19                      .edit().putBoolean(LandingActivity.KEY_PRIMERA_VEZ, false).apply();
20
21              LandingFinalFragment finalFragment = new LandingFinalFragment();
22              FragmentTransaction transaction = getActivity().getSupportFragmentManager().
        beginTransaction();
23              transaction.replace(R.id.fragmentContainerLanding, finalFragment);
24              transaction.commit();
25          }
26      });
27
28      return rootView;
29  }
```

5.10. LandingFinalFragment.java

5.10.1. Ubicación

`com.example.appnotas.View.Landing`

Descripción general:
Último paso: muestra «Registro completado con éxito» con la tortuga y, a los 3 segundos, abre la lista de notas.

5.10.2. Funciones

onCreateView **Descripción:** Infla la vista y, pasados 3 segundos, abre la pantalla general y cierra la bienvenida.

Código comentado:

```
@Override
public View onCreateView(LayoutInflater inflater, ViewGroup container,
                         Bundle savedInstanceState) {
    View rootView = inflater.inflate(R.layout.fragment_landing_final, container, false);

    new Handler().postDelayed(() -> {
        Intent intent = new Intent(getActivity(), GeneralActivity.class);
        startActivity(intent);
        getActivity().finish();
    }, 3000);

    return rootView;
}
```

5.11. GeneralActivity.java

5.11.1. Ubicación

`com.example.appnotas.View`

Descripción general:
La clase `GeneralActivity` es la pantalla principal: saluda al usuario, muestra la lista de notas, permite filtrarlas por color y tiene el menú lateral con el resto de pantallas.

5.11.2. Funciones

onCreate **Descripción:** Enlaza las vistas, carga el saludo y las notas, y prepara el filtro, la edición y el menú.

Código comentado:

```
@Override
protected void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
    EdgeToEdge.enable(this);
    setContentView(R.layout.activity_general);
    ViewCompat.setOnApplyWindowInsetsListener(findViewById(R.id.main), (v, insets) -> {
        Insets systemBars = insets.getInsets(WindowInsetsCompat.Type.systemBars());
        v.setPadding(systemBars.left, systemBars.top, systemBars.right, systemBars.bottom);
        return insets;
```

```
    });

    chip1 = findViewById(R.id.chip1);
    chip2 = findViewById(R.id.chip2);
    chip3 = findViewById(R.id.chip3);
    chip4 = findViewById(R.id.chip4);
    chip5 = findViewById(R.id.chip5);
    chip6 = findViewById(R.id.chip6);
    chip7 = findViewById(R.id.chip7);

    noNotas = findViewById(R.id.textView3);
    saludo = findViewById(R.id.textView5);
    listView = findViewById(R.id.listView);
    drawerLayout = findViewById(R.id.drawerLayout);
    menuLateral = findViewById(R.id.menuLateral);
    btnMenu = findViewById(R.id.btnMenu);

    adapter = new NotaAdapter(this, listaFiltrada);
    listView.setAdapter(adapter);

    LandingViewModel viewModel = new LandingViewModel();
    Usuario usuario = viewModel.leerUsuario(this);
    String textoSaludo = getString(R.string.saludo, usuario.getNombre());
    saludo.setText(textoSaludo);

    configurarMenuLateral(textoSaludo);

    cargarNotasDesdeArchivos();

    filtrado();

    edicion();
}
```

configurarMenuLateral **Descripción:** Abre el menú con el icono de la cabecera y lleva a Escribir nota, Opciones o Nota aleatoria. Si el menú está abierto, el botón atrás lo cierra.

Código comentado:

```
// Menú lateral (diseño de Figma): Escribir nota, Opciones y Nota aleatoria
private void configurarMenuLateral(String textoSaludo) {
    TextView saludoMenu = menuLateral.getHeaderView(0).findViewById(R.id.txtSaludoMenu);
    saludoMenu.setText(textoSaludo);

    btnMenu.setOnClickListener(v -> drawerLayout.openDrawer(GravityCompat.START));

    menuLateral.setNavigationItemSelectedListener(item -> {
        drawerLayout.closeDrawer(GravityCompat.START);
        int id = item.getItemId();
        if (id == R.id.menuEscribir) {
            abrir(CrearNotasActivity.class);
        } else if (id == R.id.menuOpciones) {
            abrir(OpcionesActivity.class);
        } else if (id == R.id.menuAleatoria) {
            abrir(NotaAleatoriaActivity.class);
        }
        return true;
    });

    // Si el menú está abierto, el botón atrás lo cierra en vez de salir de la app
    getOnBackPressedDispatcher().addCallback(this, new OnBackPressedCallback(true) {
        @Override
        public void handleOnBackPressed() {
            if (drawerLayout.isDrawerOpen(GravityCompat.START)) {
                drawerLayout.closeDrawer(GravityCompat.START);
            } else {
                finish();
```

```
            }
        }
    });
}
```

abrir **Descripción:** Abre otra pantalla y cierra esta.

Código comentado:

```
private void abrir(Class<?> pantalla) {
    startActivity(new Intent(GeneralActivity.this, pantalla));
    finish();
}
```

filtrado **Descripción:** Hace que cada chip de color vuelva a filtrar la lista al marcarse o desmarcarse.

Código comentado:

```
private void filtrado(){
    chip1.setOnCheckedChangeListener((b, c) -> filtrar());
    chip2.setOnCheckedChangeListener((b, c) -> filtrar());
    chip3.setOnCheckedChangeListener((b, c) -> filtrar());
    chip4.setOnCheckedChangeListener((b, c) -> filtrar());
    chip5.setOnCheckedChangeListener((b, c) -> filtrar());
    chip6.setOnCheckedChangeListener((b, c) -> filtrar());
    chip7.setOnCheckedChangeListener((b, c) -> filtrar());
}
```

edicion **Descripción:** Al tocar una nota, abre el formulario con sus datos para editarla.

Código comentado:

```
private void edicion() {
    listView.setOnItemClickListener((parent, view, position, id) -> {
        Nota seleccionada = listaFiltrada.get(position);

        Intent intent = new Intent(GeneralActivity.this, CrearNotasActivity.class);
        intent.putExtra("id", seleccionada.getId());
        intent.putExtra("titulo", seleccionada.getTitulo());
        intent.putExtra("descripcion", seleccionada.getDescripcion());
        intent.putExtra("color", seleccionada.getColor().name());
        startActivity(intent);
        finish();
    });
}
```

cargarNotasDesdeArchivos **Descripción:** Pide las notas al `NotaManager` y las muestra; si no hay, enseña el aviso.

Código comentado:

```
private void cargarNotasDesdeArchivos() {
    listaNotas.clear();
    listaFiltrada.clear();

    listaNotas.addAll(new NotaManager(this).leerNotas());
    listaFiltrada.addAll(listaNotas);
    adapter.notifyDataSetChanged();

    noNotas.setVisibility(listaFiltrada.isEmpty() ? View.VISIBLE : View.GONE);
}
```

filtrar **Descripción:** Muestra solo las notas de los colores marcados. Sin ningún chip marcado, o con todos, se ven todas.

Código comentado:

```
private void filtrar() {

    boolean filtroColor1 = chip1.isChecked();
    boolean filtroColor2 = chip2.isChecked();
    boolean filtroColor3 = chip3.isChecked();
    boolean filtroColor4 = chip4.isChecked();
    boolean filtroColor5 = chip5.isChecked();
    boolean filtroColor6 = chip6.isChecked();
    boolean filtroColor7 = chip7.isChecked();

    boolean ninguno = !filtroColor1 && !filtroColor2 && !filtroColor3 && !filtroColor4 && !
     filtroColor5 && !filtroColor6 && !filtroColor7;
    boolean todos = filtroColor1 && filtroColor2 && filtroColor3 && filtroColor4 && filtroColor5
     && filtroColor6 && filtroColor7;

    listaFiltrada.clear();

    if (ninguno || todos) {
        listaFiltrada.addAll(listaNotas);
    } else {
        for (int i = 0; i < listaNotas.size(); i++) {
            ColorNota color = listaNotas.get(i).getColor();

            if ((filtroColor1 && color == ColorNota.Color1) ||
                    (filtroColor2 && color == ColorNota.Color2) ||
                    (filtroColor3 && color == ColorNota.Color3) ||
                    (filtroColor4 && color == ColorNota.Color4) ||
                    (filtroColor5 && color == ColorNota.Color5) ||
                    (filtroColor6 && color == ColorNota.Color6) ||
                    (filtroColor7 && color == ColorNota.Color7)) {

                listaFiltrada.add(listaNotas.get(i));
            }
        }

    }

    noNotas.setVisibility(listaFiltrada.isEmpty() ? View.VISIBLE : View.GONE);

    adapter.notifyDataSetChanged();
}
```

5.12. CrearNotasActivity.java

5.12.1. Ubicación

`com.example.appnotas.View`

Descripción general:
La clase `CrearNotasActivity` es el formulario para escribir una nota nueva o editar una existente: título, texto y color.

5.12.2. Funciones

onCreate **Descripción:** Si llega una nota por el `Intent`, rellena los campos y cambia el título a «Editar nota».

Código comentado:

```java
@Override
protected void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
    EdgeToEdge.enable(this);
    setContentView(R.layout.activity_crear_notas);
    ViewCompat.setOnApplyWindowInsetsListener(findViewById(R.id.main), (v, insets) -> {
        Insets systemBars = insets.getInsets(WindowInsetsCompat.Type.systemBars());
        v.setPadding(systemBars.left, systemBars.top, systemBars.right, systemBars.bottom);
        return insets;
    });

    txtTituloEdit = findViewById(R.id.txtTituloEdit);
    txtDescripcionEdit = findViewById(R.id.txtDescripcionEdit);
    btnEnviar = findViewById(R.id.btnEnviar);
    btnBack = findViewById(R.id.btnBack);

    chip1 = findViewById(R.id.chip1);
    chip2 = findViewById(R.id.chip2);
    chip3 = findViewById(R.id.chip3);
    chip4 = findViewById(R.id.chip4);
    chip5 = findViewById(R.id.chip5);
    chip6 = findViewById(R.id.chip6);
    chip7 = findViewById(R.id.chip7);

    Intent intent = getIntent();
    if (getIntent().hasExtra("id")) {
        esEdicion = true;
        TextView tituloPantalla = findViewById(R.id.txtTituloPantalla);
        tituloPantalla.setText(R.string.editar_nota);
        notaId = intent.getIntExtra("id", 0);
        txtTituloEdit.setText(intent.getStringExtra("titulo"));
        txtDescripcionEdit.setText(intent.getStringExtra("descripcion"));

        String colorStr = intent.getStringExtra("color");

        ColorNota color = ColorNota.valueOf(colorStr);

        switch (color) {
            case Color1: chip1.setChecked(true); break;
            case Color2: chip2.setChecked(true); break;
            case Color3: chip3.setChecked(true); break;
            case Color4: chip4.setChecked(true); break;
            case Color5: chip5.setChecked(true); break;
            case Color6: chip6.setChecked(true); break;
            case Color7: chip7.setChecked(true); break;
        }
    }

    btnEnviar.setOnClickListener(v -> guardarNota());

    btnBack.setOnClickListener(v-> {
        startActivity(new Intent(CrearNotasActivity.this, GeneralActivity.class));
        finish();
    });

}
```

guardarNota **Descripción:** Valida que haya título o texto, guarda o actualiza la nota y avisa con un `Toast`.

Código comentado:

```java
private void guardarNota() {
    String titulo = txtTituloEdit.getText().toString().trim();
    String descripcion = txtDescripcionEdit.getText().toString().trim();

    if(titulo.isEmpty() && descripcion.isEmpty()){
        Toast.makeText(this, "Escribe un título o una descripción", Toast.LENGTH_SHORT).show();
```

```
        return;
    }

    ColorNota colorSeleccionado = ColorNota.Color1;
    if (chip1.isChecked()) colorSeleccionado = ColorNota.Color1;
    else if (chip2.isChecked()) colorSeleccionado = ColorNota.Color2;
    else if (chip3.isChecked()) colorSeleccionado = ColorNota.Color3;
    else if (chip4.isChecked()) colorSeleccionado = ColorNota.Color4;
    else if (chip5.isChecked()) colorSeleccionado = ColorNota.Color5;
    else if (chip6.isChecked()) colorSeleccionado = ColorNota.Color6;
    else if (chip7.isChecked()) colorSeleccionado = ColorNota.Color7;

    NotaManager manager = new NotaManager(this);

    if (esEdicion) {
        Nota nota = new Nota(notaId, titulo, descripcion, colorSeleccionado);
        manager.actualizarNota(nota);
        Toast.makeText(this, "Nota editada correctamente", Toast.LENGTH_SHORT).show();

        startActivity(new Intent(CrearNotasActivity.this, GeneralActivity.class));
        finish();
    } else {
        Nota nota = new Nota(0, titulo, descripcion, colorSeleccionado);
        manager.guardarNota(nota);
        Toast.makeText(this, "Nota creada", Toast.LENGTH_SHORT).show();

        // limpiar campos
        txtTituloEdit.setText("");
        txtDescripcionEdit.setText("");

        // desseleccionar chips
        chip1.setChecked(false);
        chip2.setChecked(false);
        chip3.setChecked(false);
        chip4.setChecked(false);
        chip5.setChecked(false);
        chip6.setChecked(false);
        chip7.setChecked(false);
    }
}
```

5.13. OpcionesActivity.java

5.13.1. Ubicación

`com.example.appnotas.View`

Descripción general:
La clase `OpcionesActivity` reúne las opciones de la cuenta: cambiar el nombre, eliminar todas las notas y reiniciar la app. Las dos últimas piden confirmación porque no se pueden deshacer.

5.13.2. Funciones

dialogoCambiarNombre **Descripción:** Pide el nombre nuevo en un diálogo que no se cierra si el campo está vacío.

Código comentado:

```
private void dialogoCambiarNombre() {
    View vista = getLayoutInflater().inflate(R.layout.dialogo_nombre, null);
    EditText editNombre = vista.findViewById(R.id.editNuevoNombre);

```

```
    AlertDialog dialogo = new MaterialAlertDialogBuilder(this)
            .setTitle(R.string.dialogo_nombre_titulo)
            .setView(vista)
            .setNegativeButton(R.string.cancelar, null)
            .setPositiveButton(R.string.guardar, null)
            .create();

    // El botón Guardar se configura aquí para que el diálogo no se cierre si el nombre está vací
     o
    dialogo.setOnShowListener(d -> dialogo.getButton(AlertDialog.BUTTON_POSITIVE).
     setOnClickListener(v -> {
        String nombre = editNombre.getText().toString().trim();
        if (nombre.isEmpty()) {
            editNombre.setError(getString(R.string.dialogo_nombre_vacio));
            return;
        }
        boolean ok = new LandingViewModel().cambiarNombre(this, nombre);
        dialogo.dismiss();
        if (ok) {
            avisoCorrecto(R.string.dialogo_nombre_ok);
        }
    }));
    dialogo.show();
}
```

dialogoEliminarNotas **Descripción:** Pide confirmación y borra todas las notas.

Código comentado:

```
private void dialogoEliminarNotas() {
    new MaterialAlertDialogBuilder(this)
            .setTitle(R.string.opciones_eliminar_notas)
            .setMessage(R.string.dialogo_eliminar_mensaje)
            .setNegativeButton(R.string.cancelar, null)
            .setPositiveButton(R.string.eliminar, (d, w) -> {
                new NotaManager(this).borrarTodasLasNotas();
                avisoCorrecto(R.string.dialogo_eliminar_ok);
            })
            .show();
}
```

reiniciarApp **Descripción:** Borra notas, usuario y preferencias, y vuelve a la bienvenida como la primera vez.

Código comentado:

```
// Borra notas, usuario y preferencias, y vuelve a la bienvenida como la primera vez
private void reiniciarApp() {
    new NotaManager(this).borrarTodo();
    getSharedPreferences(LandingActivity.PREFS_APP, Context.MODE_PRIVATE).edit().clear().apply();

    Toast.makeText(this, R.string.dialogo_reiniciar_ok, Toast.LENGTH_SHORT).show();

    Intent intent = new Intent(OpcionesActivity.this, LandingActivity.class);
    intent.addFlags(Intent.FLAG_ACTIVITY_NEW_TASK | Intent.FLAG_ACTIVITY_CLEAR_TASK);
    startActivity(intent);
    finish();
}
```

avisoCorrecto **Descripción:** Muestra el aviso de que todo ha ido bien, con la tortuga.

Código comentado:

```java
// Aviso con la tortuga, como en el diseño de Figma
private void avisoCorrecto(int mensaje) {
    View vista = getLayoutInflater().inflate(R.layout.dialogo_aviso, null);
    TextView texto = vista.findViewById(R.id.txtAviso);
    texto.setText(mensaje);

    new MaterialAlertDialogBuilder(this)
            .setView(vista)
            .setPositiveButton(R.string.aceptar, null)
            .show();
}
```

5.14. NotaAleatoriaActivity.java

5.14.1. Ubicación

`com.example.appnotas.View`

Descripción general:
La clase `NotaAleatoriaActivity` enseña una de las notas elegida al azar, con su color.

5.14.2. Funciones

mostrarNotaAleatoria **Descripción:** Elige una nota al azar sin repetir la que ya se está viendo; si no hay notas, muestra un aviso.

Código comentado:

```java
private void mostrarNotaAleatoria() {
    if (notas.isEmpty()) {
        contenedorNota.setVisibility(View.GONE);
        botones.setVisibility(View.GONE);
        txtSinNotas.setVisibility(View.VISIBLE);
        return;
    }

    // Si hay más de una nota, se evita repetir la que ya se está viendo
    Nota elegida;
    do {
        elegida = notas.get(random.nextInt(notas.size()));
    } while (notas.size() > 1 && elegida == notaActual);
    notaActual = elegida;

    txtTitulo.setText(notaActual.getTitulo());
    txtTexto.setText(notaActual.getDescripcion());
    contenedorNota.setBackgroundResource(NotaAdapter.fondoDe(notaActual.getColor()));
}
```

editarNota **Descripción:** Abre la nota que se está viendo en el formulario de edición.

Código comentado:

```java
private void editarNota() {
    if (notaActual == null) return;
    Intent intent = new Intent(NotaAleatoriaActivity.this, CrearNotasActivity.class);
    intent.putExtra("id", notaActual.getId());
    intent.putExtra("titulo", notaActual.getTitulo());
    intent.putExtra("descripcion", notaActual.getDescripcion());
    intent.putExtra("color", notaActual.getColor().name());
```

```
8       startActivity(intent);
9       finish();
10  }
```

5.15. NotaAdapter.java

5.15.1. Ubicación

`com.example.appnotas.View`

Descripción general:
La clase `NotaAdapter` es el adaptador del `ListView`: pinta cada nota con su título, su texto y el fondo de su color.

5.15.2. Funciones

getView **Descripción:** Rellena la fila de una nota y le pone el fondo de su color.

Código comentado:

```
1  public View getView(int position, View convertView, ViewGroup parent) {
2      View itemView = LayoutInflater.from(getContext()).inflate(R.layout.item_nota, parent, false);
3
4      Nota nota = getItem(position);
5
6      TextView titulo = itemView.findViewById(R.id.Titulo);
7      TextView descripcion = itemView.findViewById(R.id.Descripcion);
8
9      titulo.setText(nota.getTitulo());
10     descripcion.setText(nota.getDescripcion());
11
12     ConstraintLayout fondo = itemView.findViewById(R.id.itemNotaLayout);
13
14     fondo.setBackgroundResource(fondoDe(nota.getColor()));
15
16     return itemView;
17 }
```

fondoDe **Descripción:** Devuelve el fondo redondeado de cada color. También lo usa la nota aleatoria.

Código comentado:

```
1  // Fondo redondeado que corresponde a cada color (también lo usa la nota aleatoria)
2  public static int fondoDe(ColorNota color) {
3      switch (color) {
4          case Color2: return R.drawable.rectangulo_fondo_lista2;
5          case Color3: return R.drawable.rectangulo_fondo_lista3;
6          case Color4: return R.drawable.rectangulo_fondo_lista4;
7          case Color5: return R.drawable.rectangulo_fondo_lista5;
8          case Color6: return R.drawable.rectangulo_fondo_lista6;
9          case Color7: return R.drawable.rectangulo_fondo_lista7;
10         default: return R.drawable.rectangulo_fondo_lista;
11     }
12 }
```

6. Mejoras futuras

- Guardar la contraseña con un **cifrado real** (por ejemplo, un *hash*); ahora solo se desplaza cada carácter una posición.
- Poder **borrar una sola nota** desde la lista.
- Guardar las notas en una **base de datos local** (Room o SQLite) en lugar de archivos de texto.
- Cambiar el identificador `com.example.appnotas` por uno propio.